

PRACTICAL WEEK -9

Session-9

TASK 1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/time.h>

#define BUFFER_SIZE 4096

/* Low-level file copy */
double low_level_copy(const char *source, const char *destination)
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;
    struct timeval start, end;

    src = open(source, O_RDONLY);

    if (src == -1)
    {
        perror("open");
        exit(1);
    }

    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest == -1)
    {
        perror("open");
        close(src);
        exit(1);
    }

    gettimeofday(&start, NULL);

    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(dest, buffer, bytes_read);

        if (bytes_written != bytes_read)
        {
            perror("write");
            close(src);
            close(dest);
            exit(1);
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
        }
        exit(1);
    }

    /* Demonstrate lseek */
    lseek(src, 0, SEEK_SET);

    gettimeofday(&end, NULL);

    close(src);
    close(dest);

    return (end.tv_sec - start.tv_sec) +
        (end.tv_usec - start.tv_usec) / 1000000.0;
}

/* Standard library file copy */
double standard_copy(const char *source, const char *destination)
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;
    struct timeval start, end;

    src = fopen(source, "rb");

    if (src == NULL)
    {
        perror("fopen");
        exit(1);
    }

    dest = fopen(destination, "wb");

    if (dest == NULL)
    {
        perror("fopen");
        fclose(src);
        exit(1);
    }

    gettimeofday(&start, NULL);

    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
    {
        if (fwrite(buffer, 1, bytes_read, dest) != bytes_read)
            break;
    }
}
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
        {
            if (fwrite(buffer, 1, bytes_read, dest) != bytes_read)
            {
                perror("fwrite");
                fclose(src);
                fclose(dest);
                exit(1);
            }
        }

        gettimeofday(&end, NULL);

        fclose(src);
        fclose(dest);

        return (end.tv_sec - start.tv_sec) +
            (end.tv_usec - start.tv_usec) / 1000000.0;
    }

int main()
{
    const char *source = "input.txt";
    const char *low_copy = "low_level_copy.txt";
    const char *std_copy = "standard_copy.txt";

    double low_time;
    double std_time;

    printf("Session 9 - File Copy Performance\n");
    printf("-----\n");

    low_time = low_level_copy(source, low_copy);
    std_time = standard_copy(source, std_copy);

    printf("\nLow-Level I/O:\n");
    printf("open(), read(), write(), lseek(), close()\n");
    printf("Time: %.6f seconds\n", low_time);

    printf("\nStandard Library I/O:\n");
    printf("fopen(), fread(), fwrite(), fclose()\n");
    printf("Time: %.6f seconds\n", std_time);

    printf("\nFiles copied successfully.\n");
    printf("Low-level copy : %s\n", low_copy);
    printf("Standard copy : %s\n", std_copy);
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
        perror("fwrite");
        fclose(src);
        fclose(dest);
        exit(1);
    }
}

gettimeofday(&end, NULL);

fclose(src);
fclose(dest);

return (end.tv_sec - start.tv_sec) +
        (end.tv_usec - start.tv_usec) / 1000000.0;
}

int main()
{
    const char *source = "input.txt";
    const char *low_copy = "low_level_copy.txt";
    const char *std_copy = "standard_copy.txt";

    double low_time;
    double std_time;

    printf("Session 9 - File Copy Performance\n");
    printf("-----\n");

    low_time = low_level_copy(source, low_copy);
    std_time = standard_copy(source, std_copy);

    printf("\nLow-Level I/O:\n");
    printf("open(), read(), write(), lseek(), close()\n");
    printf("Time: %.6f seconds\n", low_time);

    printf("\nStandard Library I/O:\n");
    printf("fopen(), fread(), fwrite(), fclose()\n");
    printf("Time: %.6f seconds\n", std_time);

    printf("\nFiles copied successfully.\n");
    printf("Low-level copy   : %s\n", low_copy);
    printf("Standard copy    : %s\n", std_copy);

    return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session9_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ echo "This is a test file for Linux file I/O." > input.txt
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session9_task1.c -o session9_task1
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session9_task1
Session 9 - File Copy Performance
-----

Low-Level I/O:
open(), read(), write(), lseek(), close()
Time: 0.000040 seconds

Standard Library I/O:
fopen(), fread(), fwrite(), fclose()
Time: 0.000017 seconds

Files copied successfully.
Low-level copy   : low_level_copy.txt
Standard copy    : standard_copy.txt
```

TASK 2: Develop a program that redirects standard input and Output using file descriptors and dup2(). Demonstrate how shells implement 1/0 redirection.

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int input_fd;
    int output_fd;
    pid_t pid;

    printf("Session 9 - I/O Redirection using dup2()\n");
    printf("-----\n");

    /* Create input file */
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1)
    {
        perror("Error opening input.txt");
        return 1;
    }

    /* Create output file */
    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd == -1)
    {
        perror("Error opening output.txt");
        close(input_fd);
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        close(input_fd);
        close(output_fd);
        return 1;
    }

    if (pid == 0)
    {
        /* Redirect standard input */
        if (dup2(input_fd, STDIN_FILENO) == -1)
        {
            perror("dup2 stdin");
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV X + v
GNU nano 8.7.1
/* Redirect standard input */
if (dup2(input_fd, STDIN_FILENO) == -1)
{
    perror("dup2 stdin");
    exit(1);
}

/* Redirect standard output */
if (dup2(output_fd, STDOUT_FILENO) == -1)
{
    perror("dup2 stdout");
    exit(1);
}

close(input_fd);
close(output_fd);

printf("This output is redirected to output.txt\n");

/* Read from redirected stdin */
char buffer[100];

ssize_t n = read(STDIN_FILENO,
                 buffer,
                 sizeof(buffer) - 1);

if (n > 0)
{
    buffer[n] = '\0';

    printf("Data read from input.txt:\n");
    printf("%s\n", buffer);
}

exit(0);
}
else
{
    close(input_fd);
    close(output_fd);

    wait(NULL);

    printf("Parent process completed.\n");
    printf("Standard input was redirected from input.txt\n");
    printf("Standard output was redirected to output.txt\n");
    printf("I/O redirection completed successfully.\n");
}

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session9_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ echo "Hello from the input file." > input.txt
keerthi_priya@LAPTOP-OSGVCE7A:~$ cat input.txt
Hello from the input file.
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session9_task2.c -o session9_task2
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session9_task2
Session 9 - I/O Redirection using dup2()
-----
Parent process completed.
Standard input was redirected from input.txt
Standard output was redirected to output.txt
I/O redirection completed successfully.
keerthi_priya@LAPTOP-OSGVCE7A:~$ cat output.txt
This output is redirected to output.txt
Data read from input.txt:
Hello from the input file.
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```